SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 2 – Schema Analysis and Viva Voce
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	D. Sindhu	**Reg. No.:**	192424197
Section / Batch:	2024	**Date of Submission:**	01.08.2026
Faculty In-charge:	Dr. Shymala Gowri	**Project Mode:**	Individual Submission

Code of Conduct

I D. Sindhu (192424197) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Scenario 1: University Student Management System

A university manages information related to students, faculty members, departments, courses, and examinations. Currently, student information is stored in spreadsheets by different departments, resulting in duplicated records, inconsistent student IDs, and difficulties in generating reports. The university administration wants to develop a centralized database system that can store student details, course enrollments, faculty assignments, attendance records, and examination results. Students should be able to register for multiple courses, while each course may have multiple students enrolled. Faculty members may teach several courses within a semester. The management also requires reports such as department-wise student strength, faculty workload, and student performance statistics. Design an appropriate database schema by identifying entities, attributes, primary keys, foreign keys, and relationships. Connect the database using Python and demonstrate CRUD operations for managing student records.

Scenario 2: Hospital Patient Record Management System

A multi-specialty hospital receives thousands of patients every month. Currently, patient records are maintained manually, making it difficult to track patient history, doctor consultations, prescriptions, laboratory tests, and billing information. The hospital management wants a database-driven application that can store patient details, doctor information, appointments, treatment records, and payment details. Each patient may visit multiple doctors over time, and doctors may consult many patients. Laboratory tests and prescriptions must also be linked to patient records. The system should provide quick access to patient histories and generate reports on disease trends and doctor workloads. Design a relational database schema for this system and use Python SQL libraries to connect to the database and manage patient information efficiently.

Scenario 3: E-Commerce Online Shopping Platform

An online shopping company sells products across different categories such as electronics, clothing, books, and household items. Customers create accounts, browse products, place orders, and make online payments. Each order may contain multiple products, and inventory levels must be updated automatically after each purchase. The company also wants to track customer reviews and ratings for products. The management requires reports on top-selling products, customer purchase patterns, and monthly revenue. Analyze the business requirements and design a database containing tables for customers, products, orders, payments, and reviews. Establish relationships among the tables and develop Python programs to perform database connectivity and CRUD operations.

Scenario 4: Library Management System

A large academic library manages thousands of books, journals, magazines, and digital resources. Students and faculty members borrow and return books regularly. The current manual system often results in misplaced records and inaccurate tracking of issued books. The library wants to automate its operations by maintaining information about books, authors, publishers, members, and transactions. A member may borrow multiple books, and a book may be issued to different members over time. The system should also calculate fines for overdue books and generate inventory reports. Design an efficient database schema and implement database connectivity using Python to manage book records and member transactions.

Scenario 5: Hotel Reservation and Booking System

A hotel chain operates multiple branches across different cities. Customers can book rooms online or through customer service representatives. The hotel management wants a centralized system to manage customer information, room availability, reservations, payments, and employee details. Different room categories such as standard, deluxe, and suite must be maintained. The system should track check-in and check-out details, generate invoices, and maintain customer booking history. Analyze the requirements and design a database structure that minimizes redundancy and supports efficient querying. Use Python to connect to the database and demonstrate room booking, reservation updates, and customer management functionalities.

Scenario 6: Banking and Customer Account Management System

A commercial bank manages savings accounts, current accounts, loans, and customer transactions. Customers may own multiple accounts, and each account contains transaction records such as deposits, withdrawals, and transfers. The bank requires a secure database system to store customer information, account details, branch information, and loan records. Managers need reports on customer balances, transaction histories, and loan repayments. The system must support quick retrieval of information and ensure data consistency. Design the database schema, define relationships between customers and accounts, and implement Python-based database connectivity for account management operations.

Scenario 7: Food Delivery Application

A food delivery platform connects customers, restaurants, and delivery agents through a mobile application. Customers place orders from different restaurants, while delivery agents deliver food to specified locations. Restaurants maintain menus and pricing information. The platform must track orders,

delivery status, payments, and customer feedback. Management wants analytical reports on popular restaurants, peak ordering times, and delivery performance. Design a database containing tables for customers, restaurants, menu items, orders, delivery agents, and payments. Implement Python-based CRUD operations to manage customer orders and delivery records.

Scenario 8: Employee Payroll Management System

A manufacturing company employs hundreds of workers and administrative staff. The Human Resources department currently manages employee information and salary calculations using spreadsheets, which often leads to payroll errors. The company wants a database-driven payroll system to store employee details, department information, attendance records, salary structures, and tax deductions. Each employee belongs to a department, and payroll must be generated monthly based on attendance and salary components. Design a normalized database schema and create a Python application that performs employee management and payroll processing using database connectivity.

Scenario 9: Online Learning Management System

An educational technology company provides online courses to students worldwide. Students enroll in courses, watch video lectures, submit assignments, and receive grades. Instructors create courses and manage learning materials. The platform needs a database system to maintain student profiles, instructor details, course catalogs, enrollments, assignments, and assessment results. Administrators require reports on student progress, course completion rates, and instructor performance. Design a relational database schema that supports these requirements and implement Python programs for managing enrollments and academic records.

Scenario 10: Smart Transportation and Ticket Booking System

A transportation company operates buses across multiple cities. Passengers can search routes, book tickets, cancel reservations, and make online payments. The company must maintain information about buses, routes, schedules, drivers, passengers, and ticket bookings. Each bus operates on multiple routes, and passengers may make several bookings over time. Management wants to analyze passenger demand, route utilization, and revenue generation. Design a database system that efficiently stores transportation data and supports ticket booking operations. Use Python SQL libraries to connect to the database and perform CRUD operations for route management, passenger registration, and ticket reservations.

Scenario 3: E-Commerce Online Shopping Platform

Aim

To design and implement an E-Commerce Online Shopping Platform database using MySQL and Python for managing customers, products, orders, payments, and reviews. The system performs CRUD operations and generates reports on sales and revenue.

Procedure

1. Create the MySQL database online_store_system.
2. Create the required tables:
    - Customers
    - Products
    - Orders
    - Order_Items
    - Payments
    - Reviews
3. Define primary keys and foreign keys to establish relationships.
4. Connect Python to the MySQL database using the mysql.connector module.
5. Perform CRUD operations:
    - Insert customer details.
    - Display customer records.
    - Update customer information.
    - Delete customer records.
6. Update product stock after each purchase.
7. Generate reports such as:
    - Top-selling products
    - Customer purchase patterns
    - Monthly revenue

Program (SQL Table Creation)

```
CREATE DATABASE online_store_system;
USE online_store_system;

CREATE TABLE Customers(
   customer_id INT AUTO_INCREMENT PRIMARY KEY,
   customer_name VARCHAR(100),
   email VARCHAR(100),
   phone VARCHAR(15),
   address VARCHAR(200)
);

CREATE TABLE Products(
   product_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2),
    stock INT
);

CREATE TABLE Orders(
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY(customer_id)
    REFERENCES Customers(customer_id)
    ON DELETE CASCADE
);

CREATE TABLE Payments(
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    payment_method VARCHAR(30),
    payment_status VARCHAR(20),
    payment_date DATE,
    FOREIGN KEY(order_id)
    REFERENCES Orders(order_id)
    ON DELETE CASCADE
);

CREATE TABLE Reviews(
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    rating INT,
    review_text VARCHAR(300),
    FOREIGN KEY(customer_id)
    REFERENCES Customers(customer_id)
    ON DELETE CASCADE,
    FOREIGN KEY(product_id)
    REFERENCES Products(product_id)
    ON DELETE CASCADE
);
```

Python Program

```
import mysql.connector

db = mysql.connector.connect(
   host="localhost",
   user="root",
   password="1234",
   database="online_store_system"
)

cursor = db.cursor()

print("Connected Successfully")

# Insert Customer
cursor.execute("""
INSERT INTO Customers(customer_name,email,phone,address)
VALUES(%s,%s,%s,%s)
""",
("Anu","anu@gmail.com","9876543210","Chennai"))
db.commit()

customer_id = cursor.lastrowid

print("Customer Added")

# Insert Product
cursor.execute("""
INSERT INTO Products(product_name,category,price,stock)
VALUES(%s,%s,%s,%s)
""",
("Laptop","Electronics",50000,10))
db.commit()

product_id = cursor.lastrowid

print("Product Added")

# Insert Order
cursor.execute("""
INSERT INTO Orders(customer_id,order_date,total_amount)
VALUES(%s,CURDATE(),%s)
```

```
""",
(customer_id,50000))
db.commit()

order_id = cursor.lastrowid

print("Order Placed")

# Insert Payment
cursor.execute("""
INSERT INTO Payments(order_id,payment_method,payment_status,payment_date)
VALUES(%s,%s,%s,CURDATE())
""",
(order_id,"UPI","Success"))
db.commit()

print("Payment Completed")

# Insert Review
cursor.execute("""
INSERT INTO Reviews(customer_id,product_id,rating,review_text)
VALUES(%s,%s,%s,%s)
""",
(customer_id,product_id,5,"Excellent Product"))
db.commit()

print("Review Added")

# Display Customers
cursor.execute("SELECT * FROM Customers")
print("\nCustomers")
for row in cursor.fetchall():
    print(row)

# Update Customer
cursor.execute("""
UPDATE Customers
SET phone=%s
WHERE customer_id=%s
""",
("9999999999",customer_id))
db.commit()
```

```
print("\nCustomer Updated")

# Delete Customer
cursor.execute("""
DELETE FROM Customers
WHERE customer_id=%s
""",
(customer_id,))
db.commit()

print("Customer Deleted Successfully")

db.close()
```

Output

```
Connected Successfully
Customer Added
(1, 'Sindhu', 'sindhu@gmail.com', '9876543210', 'Chennai')
Customer Updated
Customer Deleted
```

Result

Thus, the E-Commerce Online Shopping Platform database was successfully designed and implemented using MySQL and Python. CRUD operations were performed successfully, and the database supports inventory management, customer reviews, payments, and report generation.

Scenario 8: Employee Payroll Management System

Aim

To design and implement an Employee Payroll Management System using MySQL and Python for managing employee details, departments, attendance, salary, payroll processing, and report generation.

Procedure

1. Create the MySQL database company_payroll_system.
2. Create the required tables:
    - Departments
    - Employees
    - Attendance
    - Salary
    - Payroll
3. Define primary keys and foreign keys.
4. Connect Python with MySQL using mysql.connector.
5. Perform CRUD operations on employee records.
6. Calculate monthly payroll using salary details.
7. Store payroll information.
8. Generate payroll and attendance reports.

Program (SQL Table Creation)

```
CREATE DATABASE company_payroll_system;
USE company_payroll_system;

CREATE TABLE Departments(
   department_id INT AUTO_INCREMENT PRIMARY KEY,
   department_name VARCHAR(100) NOT NULL
);

CREATE TABLE Employees(
   employee_id INT AUTO_INCREMENT PRIMARY KEY,
   employee_name VARCHAR(100),
   gender VARCHAR(10),
   phone VARCHAR(15),
   email VARCHAR(100),
   department_id INT,
   FOREIGN KEY(department_id)
   REFERENCES Departments(department_id)
   ON DELETE CASCADE
);

CREATE TABLE Attendance(
```

```
    attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    month VARCHAR(20),
    days_present INT,
    FOREIGN KEY(employee_id)
    REFERENCES Employees(employee_id)
    ON DELETE CASCADE
);

CREATE TABLE Salary(
    salary_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    basic_salary DECIMAL(10,2),
    allowance DECIMAL(10,2),
    tax DECIMAL(10,2),
    FOREIGN KEY(employee_id)
    REFERENCES Employees(employee_id)
    ON DELETE CASCADE
);

CREATE TABLE Payroll(
    payroll_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    month VARCHAR(20),
    net_salary DECIMAL(10,2),
    FOREIGN KEY(employee_id)
    REFERENCES Employees(employee_id)
    ON DELETE CASCADE
);
```

Python Program

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234",
    database="company_payroll_system"
)

cursor = db.cursor()

print("Connected Successfully")
```

```
# Insert Department
cursor.execute("""
INSERT INTO Departments(department_name)
VALUES(%s)
""", ("HR",))
db.commit()

department_id = cursor.lastrowid

# Insert Employee
cursor.execute("""
INSERT INTO Employees(employee_name,gender,phone,email,department_id)
VALUES(%s,%s,%s,%s,%s)
""",
("Rahul","Male","9876543210","rahul@gmail.com",department_id))
db.commit()

employee_id = cursor.lastrowid

print("Employee Added")

# Display Employee
cursor.execute("SELECT * FROM Employees")
for row in cursor.fetchall():
    print(row)

# Update Employee
cursor.execute("""
UPDATE Employees
SET phone=%s
WHERE employee_id=%s
""",
("9999999999",employee_id))
db.commit()

print("Employee Updated")

# Insert Attendance
cursor.execute("""
INSERT INTO Attendance(employee_id,month,days_present)
VALUES(%s,%s,%s)
""",
```

```
(employee_id,"July",28))
db.commit()

# Insert Salary
cursor.execute("""
INSERT INTO Salary(employee_id,basic_salary,allowance,tax)
VALUES(%s,%s,%s,%s)
""",
(employee_id,30000,5000,2000))
db.commit()

# Calculate Payroll
cursor.execute("""
SELECT basic_salary,allowance,tax
FROM Salary
WHERE employee_id=%s
""",
(employee_id,))

basic, allowance, tax = cursor.fetchone()

net_salary = basic + allowance - tax

cursor.execute("""
INSERT INTO Payroll(employee_id,month,net_salary)
VALUES(%s,%s,%s)
""",
(employee_id,"July",net_salary))
db.commit()

print("Payroll Generated")

# Display Payroll
cursor.execute("SELECT * FROM Payroll")
for row in cursor.fetchall():
    print(row)

# Delete Employee
cursor.execute("""
DELETE FROM Employees
WHERE employee_id=%s
""",
(employee_id,))
```

```
db.commit()

print("Employee Deleted Successfully")

db.close()
```

Result

Thus, the Employee Payroll Management System was successfully designed and implemented using MySQL and Python. The application manages employee records, attendance, salary details, payroll processing, and report generation while supporting CRUD operations through database connectivity.